

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot Payloads - XML and JSON

Table of Contents

Learning Outcome:	3
1. Introduction to Spring Boot Payloads	4
XML (Extensible Markup Language)	4
JSON (JavaScript Object Notation)	4
Importance of Payload Formats in Web Applications	4
Comparison between XML and JSON Payloads	5
2. Enabling CORS (Cross-Origin Resource Sharing) in Controller Method	6
Configuring CORS for a specific controller method in Spring Boot	6
Different CORS configuration options and their impact	7
3. Global CORS configuration in Spring Boot	8
Configuring global CORS settings using annotations and configuration classes	8
Handling CORS preflight requests and customizing CORS behavior globally	10
4. Spring Boot with XML Payload	11
Marshalling and unmarshalling XML data using JAXB	12
Configuring XML message converters in Spring Boot	14
5. Spring Boot with JSON Payload	16
Serializing and deserializing JSON data in Spring Boot using Jackson library	16
Customizing JSON serialization and deserialization behavior	18

UNIT 10: Spring Boot Payloads - XML and JSON

Objectives:

In this Unit you will learn to –

- Understand the concept of payload formats: XML and JSON, and their significance in web applications.
- Implement CORS (Cross-Origin Resource Sharing) in Spring Boot controller methods.
- Configure global CORS settings in a Spring Boot application.
- Work with XML payloads in Spring Boot applications.
- Work with JSON payloads in Spring Boot applications.

Learning Outcome:

Learn about Spring Boot Payloads

10. Spring Boot Payloads - XML and JSON

1. Introduction to Spring Boot Payloads

In modern web development, communication between client and server often involves exchanging data in different formats. Two commonly used payload formats in Spring Boot applications are XML and JSON. These formats play a crucial role in transmitting and representing data between the client and the server.

XML (Extensible Markup Language)

XML is a markup language that uses tags to define the structure and content of data. It provides a flexible and self-descriptive format for representing hierarchical data. XML is widely used in various industries and has been a popular choice for data interchange between systems for many years.

JSON (JavaScript Object Notation)

JSON is a lightweight data interchange format that has gained significant popularity in recent years. It is a text-based format that uses key-value pairs to represent data objects. JSON is easy to read, write, and parse, making it a preferred choice for many web APIs. JSON's simplicity and compatibility with JavaScript have made it a standard format for data transmission in modern web applications.

Importance of Payload Formats in Web Applications

The choice of payload format in web applications can impact various aspects, including data readability, performance, interoperability, and ease of development. The payload format determines how data is structured, transmitted, and interpreted by client and server components. Here are a few reasons why payload formats are important in web applications:

1. **Data Representation:** Payload formats define how data is represented, structured, and organized. They ensure consistency and allow systems to interpret and process data accurately.
2. **Interoperability:** Payload formats enable communication and data exchange between different systems and platforms. They provide a common language for data interchange, allowing systems built with different technologies to interact seamlessly.
3. **Readability:** Payload formats should be human-readable and easy to understand for developers during development, debugging, and testing. They should also facilitate data inspection and troubleshooting.
4. **Performance:** The payload format can impact the performance of data transmission and processing. Some formats may introduce unnecessary overhead, leading to increased bandwidth usage and slower data transfer. Choosing an efficient payload format can help optimize performance.

Comparison between XML and JSON Payloads

XML and JSON are both widely used payload formats, but they have distinct characteristics. Here's a comparison between XML and JSON payloads:

1. **Syntax:** XML uses tags and attributes to structure data hierarchically, while JSON utilizes key-value pairs organized in a more compact and straightforward structure.
2. **Readability:** XML tends to be more verbose and can be challenging to read, especially for complex structures. JSON, on the other hand, is more concise and easier to read and write for both humans and machines.
3. **Data Types:** XML supports a broader range of data types, including text, numbers, dates, and more. JSON has a limited set of data types, such as strings, numbers, booleans, arrays, and objects.
4. **Flexibility:** XML allows complex data structures and provides extensibility through namespaces and custom schemas. JSON is simpler and more suitable for representing lightweight data structures.

5. Tooling and Support: XML has been around for a longer time and has extensive tooling and support in various programming languages. JSON is widely supported in modern web frameworks and has become the de facto standard for many APIs.

When choosing between XML and JSON payloads in Spring Boot applications, it's important to consider factors such as the complexity of data, interoperability requirements, performance considerations, and the existing ecosystem of tools and libraries.

2. Enabling CORS (Cross-Origin Resource Sharing) in Controller Method

Cross-Origin Resource Sharing (CORS) is a security mechanism implemented in web browsers that governs how resources (such as fonts, scripts, or API requests) are shared across different domains or origins. It is an essential aspect of web development as it ensures that web applications can securely interact with resources from different origins while protecting users from potential malicious activities.

The Same-Origin Policy (SOP) is a fundamental security feature of web browsers that restricts web pages from making requests to a different origin than the one that served the page. CORS relaxes this restriction by defining a set of headers and rules that allow controlled cross-origin requests.

Configuring CORS for a specific controller method in Spring Boot

In Spring Boot, you can configure CORS for a specific controller method using the `@CrossOrigin` annotation. By applying this annotation to a controller method, you can define the specific origins, HTTP methods, headers, and other properties that are allowed to access the method's endpoint.

Here's an example of how to enable CORS for a specific controller method in Spring Boot:

```

@RestController

public class MyController {

    @CrossOrigin(origins = "http://example.com", methods =
RequestMethod.GET)

    @GetMapping("/api/data")

    public String getData() {

        // Method implementation

    }

}

```

In the above example, the `@CrossOrigin` annotation is used to allow cross-origin requests from the `http://example.com` origin, specifically for the GET HTTP method. This means that the `getData()` method can be accessed from `http://example.com` using a GET request.

Different CORS configuration options and their impact

The `@CrossOrigin` annotation provides several attributes that allow you to configure CORS behavior for a specific controller method. Some commonly used attributes include:

- **origins:** Specifies the allowed origins for cross-origin requests. It accepts a comma-separated list of origins or `*` to allow all origins. For example: `@CrossOrigin(origins = "http://example.com")`.
- **methods:** Defines the allowed HTTP methods for cross-origin requests. By default, all methods are allowed. You can specify specific methods using `RequestMethod` enums or `*` for all methods. For example: `@CrossOrigin(methods = RequestMethod.GET)`.
- **allowedHeaders:** Specifies the allowed request headers for cross-origin requests. It accepts a comma-separated list of headers or `*` to allow all headers.

- **exposedHeaders:** Defines the custom headers that are exposed in the response to cross-origin requests. Similar to **allowedHeaders**, it accepts a comma-separated list of headers or ***** for all headers.
- **allowCredentials:** Specifies whether the browser should include credentials (such as cookies or authorization headers) in cross-origin requests. It accepts a boolean value (true or false).
- **maxAge:** Sets the maximum time (in seconds) that the preflight request can be cached by the browser.

These attributes provide fine-grained control over CORS configuration for a specific controller method, allowing you to define the origins, methods, headers, and other properties that are allowed or restricted.

3. Global CORS configuration in Spring Boot

In addition to configuring CORS for specific controller methods, Spring Boot also provides the capability to configure CORS globally for all endpoints in the application. Global CORS configuration allows you to define default CORS settings that are applied to all requests, reducing the need to repeat the configuration for each individual controller method.

Configuring global CORS settings using annotations and configuration classes

Spring Boot offers multiple ways to configure global CORS settings. Two commonly used approaches are:

1. Using the **@CrossOrigin** annotation on a global configuration class:

You can create a global configuration class and apply the **@CrossOrigin** annotation to it. This annotation allows you to specify the default CORS settings that will be applied to all requests in the application.


```

@Configuration
@CrossOrigin(origins = "http://example.com")
public class GlobalCorsConfiguration {
}

```

In the above example, the `@CrossOrigin` annotation is applied at the class level, specifying that requests from `http://example.com` origin are allowed for all endpoints.

2. Implementing `WebMvcConfigurer` interface and overriding `addCorsMappings` method:

Alternatively, you can implement the `WebMvcConfigurer` interface and override the `addCorsMappings` method to define global CORS configuration.

```

@Configuration
public class GlobalCorsConfiguration implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://example.com")
            .allowedMethods("GET", "POST")
            .allowedHeaders("Authorization")
            .allowCredentials(true)
            .maxAge(3600);
    }
}

```

In this approach, the `addCorsMappings` method is overridden to configure CORS settings. The `CorsRegistry` provides methods to define allowed origins, methods, headers, credentials, and other CORS properties.

Handling CORS preflight requests and customizing CORS behavior globally

When making certain types of cross-origin requests, such as requests with non-simple HTTP methods (e.g., PUT, DELETE) or requests with custom headers, the browser sends a preflight request to the server to determine if the actual request is safe to send. Spring Boot provides options to handle CORS preflight requests and customize CORS behavior globally.

To handle preflight requests, you can add the OPTIONS HTTP method to the allowed methods list in the global CORS configuration. For example:

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**")
        .allowedOrigins("http://example.com")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("Authorization")
        .allowCredentials(true)
        .maxAge(3600);
}
```

Additionally, you can customize CORS behavior globally by implementing the CorsConfigurationSource interface and providing custom logic. This allows you to dynamically determine CORS settings based on various factors such as request attributes, user roles, or specific endpoints. To use a custom CorsConfigurationSource, you need to create a bean of CorsFilter with the custom source.

```
@Configuration
public class GlobalCorsConfiguration {
```

```

@Bean

public CorsFilter corsFilter() {

    UrlBasedCorsConfigurationSource source = new
    UrlBasedCorsConfigurationSource();

    CorsConfiguration config = new CorsConfiguration();

    // Customize CORS configuration based on your requirements

    // ...

    source.registerCorsConfiguration("/**", config);

    return new CorsFilter(source);

}
}

```

By customizing the CorsConfigurationSource, you have greater control over the CORS behavior in your Spring Boot application.

It's important to strike a balance between providing necessary access to resources and ensuring security. Carefully consider the CORS settings, allowing only trusted origins, methods, and headers, to mitigate potential security risks.

4. Spring Boot with XML Payload

Spring Boot provides robust support for working with XML payloads, allowing you to send and receive data in XML format. XML payloads are commonly used in various industries and systems for data interchange.

To work with XML payloads in Spring Boot, you need to perform the following tasks:

1. Define XML mappings: XML mappings define the structure and mapping of XML elements to Java objects. This is typically done using JAXB (Java Architecture for XML Binding) annotations.
2. Marshal XML data: Marshaling is the process of converting Java objects into XML representations. It involves transforming Java object properties into XML elements and attributes.

3. Unmarshal XML data: Unmarshaling is the process of converting XML data into Java objects. It involves parsing XML documents and mapping XML elements to Java object properties.

4. Configure XML message converters: XML message converters handle the conversion between XML data and Java objects. Spring Boot provides built-in XML message converters that can automatically marshal and unmarshal XML payloads.

Marshalling and unmarshalling XML data using JAXB

JAXB (Java Architecture for XML Binding) is a standard Java API for mapping Java objects to XML representations and vice versa. It simplifies the process of working with XML data by providing annotations to define XML mappings.

Here's an example of how to work with XML payloads using JAXB in Spring Boot:

1. Define XML mappings:

Create a Java class and annotate it with JAXB annotations to define the XML structure and mappings.

```
import javax.xml.bind.annotation.*;
```

```
@XmlRootElement(name = "book")
```

```
public class Book {
```

```
    private String title;
```

```
    private String author;
```

```
    @XmlElement
```

```
    public String getTitle() {
```

```
        return title;
```

```
    }
```

```

    public void setTitle(String title) {
        this.title = title;
    }

```

```

@XmlElement
    public String getAuthor() {
        return author;
    }

```

```

    public void setAuthor(String author) {
        this.author = author;
    }
}

```

2. Marshal XML data:

To marshal a Java object into XML, you can use the `JAXBContext` and `Marshaller` classes.

```

import javax.xml.bind.*;

public class XmlMarshallingExample {
    public static void main(String[] args) throws JAXBException {
        Book book = new Book();
        book.setTitle("Spring Boot in Action");
        book.setAuthor("John Smith");

        JAXBContext jaxbContext = JAXBContext.newInstance(Book.class);
        Marshaller marshaller = jaxbContext.createMarshaller();
        marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
    }
}

```

```
marshaller.marshal(book, System.out);
```

```
}
```

```
}
```

In the above example, the `JAXBContext` is created for the `Book` class, and the `Marshaller` is used to marshal the `book` object into XML.

3. Unmarshal XML data:

To unmarshal XML data into Java objects, you can use the `Unmarshaller` class.

```
import javax.xml.bind.*;
```

```
public class XmlUnmarshallingExample {
```

```
    public static void main(String[] args) throws JAXBException {
```

```
        String xml = "<book><title>Spring Boot in  
Action</title><author>John Smith</author></book>";
```

```
        JAXBContext jaxbContext = JAXBContext.newInstance(Book.class);
```

```
        Unmarshaller unmarshaller = jaxbContext.createUnmarshaller();
```

```
        Book book = (Book) unmarshaller.unmarshal(new StringReader(xml));
```

```
        System.out.println("Title: " + book.getTitle());
```

```
        System.out.println("Author: " + book.getAuthor());
```

```
}
```

```
}
```

In the above example, the `Unmarshaller` is used to unmarshal the XML string into a `Book` object.

Configuring XML message converters in Spring Boot

Spring Boot automatically configures XML message converters by default. When

Proprietary content. All rights reserved. Unauthorized use or distribution prohibited.

XML payloads are sent or received, Spring Boot uses the `'Jaxb2RootElementHttpMessageConverter'` to marshal and unmarshal XML data.

However, if you want to customize the XML message converters or provide additional configuration, you can do so by creating a `'WebMvcConfigurer'` bean and overriding the `'extendMessageConverters'` method.

```
import org.springframework.context.annotation.Configuration;

import org.springframework.http.converter.HttpMessageConverter;

import
    org.springframework.http.converter.xml.MappingJackson2XmlHttpMessageConvert
    er;

import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.util.List;

@Configuration

public class XmlMessageConverterConfig implements WebMvcConfigurer {

    @Override

    public void extendMessageConverters(List<HttpMessageConverter<?>>
        converters) {

        converters.add(new MappingJackson2XmlHttpMessageConverter());

        // Add additional XML message converters if needed

    }

}
```

In the above example, the `'extendMessageConverters'` method is overridden to add the `'MappingJackson2XmlHttpMessageConverter'` to the list of message converters. This converter allows you to work with XML payloads using Jackson's XML data-binding capabilities.

By customizing the XML message converters, you can control the XML marshaling and unmarshaling behavior, configure additional XML-related libraries, or support different XML formats.

Remember to add the necessary dependencies for JAXB and any other XML-related libraries you may require in your `pom.xml` or `build.gradle` file.

With these steps, you can effectively work with XML payloads in Spring Boot applications, leveraging JAXB annotations for XML mappings and configuring XML message converters to marshal and unmarshal XML data.

5. Spring Boot with JSON Payload

JSON (JavaScript Object Notation) is a lightweight data interchange format widely used in modern web applications. It provides a simple and human-readable structure for representing data, making it easy to transmit and consume data between different systems.

JSON consists of key-value pairs and supports various data types such as strings, numbers, booleans, arrays, and objects. It has become the de facto standard for data exchange in web APIs and is supported by most programming languages and frameworks.

Serializing and deserializing JSON data in Spring Boot using Jackson library

Spring Boot leverages the Jackson library as the default JSON library for serializing and deserializing JSON data. Jackson provides powerful features and flexibility for working with JSON in Java.

To serialize Java objects into JSON and deserialize JSON into Java objects in a Spring Boot application, you can follow these steps:

1. Include Jackson dependency:

Ensure that the Jackson library is included as a dependency in your project. Spring Boot's starters usually include the necessary Jackson dependencies by default.

2. Serialize Java objects into JSON:

To serialize Java objects into JSON, you can use the `ObjectMapper` class from the Jackson library.

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class JsonSerializationExample {

    public static void main(String[] args) throws JsonProcessingException {

        ObjectMapper objectMapper = new ObjectMapper();

        Book book = new Book("Spring Boot in Action", "John Smith");

        String json = objectMapper.writeValueAsString(book);

        System.out.println(json);

    }

}
```

In the above example, the `ObjectMapper` is used to serialize the `Book` object into a JSON string.

3. Deserialize JSON into Java objects:

To deserialize JSON into Java objects, you can use the `ObjectMapper` class.

```
import com.fasterxml.jackson.databind.ObjectMapper;

public class JsonDeserializationExample {

    public static void main(String[] args) throws JsonProcessingException {

        String json = "{\"title\":\"Spring Boot in Action\", \"author\":\"John Smith\"}";


```

```

    ObjectMapper objectMapper = new ObjectMapper();

    Book book = objectMapper.readValue(json, Book.class);

    System.out.println("Title: " + book.getTitle());

    System.out.println("Author: " + book.getAuthor());

}
}

```

In the above example, the `ObjectMapper` is used to deserialize the JSON string into a `Book` object.

Customizing JSON serialization and deserialization behavior

Jackson provides various annotations and configuration options to customize the JSON serialization and deserialization behavior in Spring Boot applications.

1. Annotations:

- `@JsonProperty`: Specifies the name of the JSON property during serialization and deserialization.
- `@JsonFormat`: Defines the date and time format for date properties.
- `@JsonIgnore`: Excludes a property from JSON serialization and deserialization.
- Many more annotations are available to fine-tune the JSON representation of Java objects.

2. Configuration options:

- **ObjectMapper configuration:** The `ObjectMapper` class provides methods to configure the serialization and deserialization behavior, such as setting date formats, enabling/disabling features, configuring custom serializers and deserializers, and more.
- **Global configuration:** You can create a bean of `Jackson2ObjectMapperBuilder` and customize it with your desired

configuration. This global configuration will be applied to all JSON serialization and deserialization operations in your Spring Boot application.

```
import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

import org.springframework.http.converter.json.Jackson2ObjectMapperBuilder;

@Configuration

public class JsonConfiguration {

    @Bean

    public Jackson2ObjectMapperBuilder jacksonBuilder() {

        Jackson2ObjectMapperBuilder builder = new
        Jackson2ObjectMapperBuilder();

        // Customize the Jackson ObjectMapper here

        return builder;

    }

}
```

In the above example, a custom 'Jackson2ObjectMapperBuilder' bean is created, allowing you to customize the Jackson 'ObjectMapper' with your desired configuration.

By utilizing Jackson's annotations and configuration options, you can customize the JSON serialization and deserialization behavior according to your application's requirements in a Spring Boot environment.